# Auth Gate 11 — User Stories (Credentials MVP)

**Status:** business analysis, ready for PM gate breakdown **Source architecture:** `docs/planning/architecture/auth-c-d.md` (locked) **Source backend:** `backend/accounts/views.py` (no changes required this gate) **In scope:** NextAuth Credentials, `/login`, `/signup`, `/verify`, `/account`, JWT-bound tier + DRF token, SSR Authorization forwarding, real-session access to `/dashboard/coordinator`. **Out of scope:** ORCID, MFA, magic-link, account linking, SSO, `COORDINATOR_API_TOKEN` deprecation.

This BA agrees with the architecture spec on all material points. The one gap to flag for the PM: the spec describes the coordinator dashboard SSR continuing to use `COORDINATOR_API_TOKEN` "for now." The user's gate scope for this session says the dashboard becomes accessible via real session in the **default** path with the service token as **emergency fallback**. The stories below follow the user's scope (real session is default); the PM should reconcile and lock this in the gate spec.

---

## Story 1 — Researcher signs up, verifies, and logs in

**As a** prospective researcher visiting `malagasyfishes.org` **I want** to create an account using my email and a password, verify via email, and sign in **So that** I can browse occurrence datasets and submit observations under a Tier 2 account

**Acceptance criteria**

1. **Given** an anonymous visitor on `/signup`, **when** they submit a valid email, name, and password, **then** the Next.js server POSTs to `/api/v1/auth/register/`, the user is created with `is_active=False` and `access_tier=2`, and the page renders a "Check your email" interstitial without revealing whether the email already existed.
2. **Given** a user clicks the verification link in their email within 48 hours, **when** the `/verify` page POSTs the token to `/api/v1/auth/verify/`, **then** the account is activated and the user is redirected to `/login?verified=1` with a success banner.
3. **Given** a verified user submits correct credentials on `/login`, **when** NextAuth's Credentials provider calls `/api/v1/auth/login/`, **then** a session cookie is set, `tier=2` and `drfToken` are stored on the JWT, and the user lands on the `callbackUrl` (or `/` if absent or not on the allow-list).
4. **Given** a verified Tier 2 user is logged in, **when** they visit `/account`, **then** they see their email, name, tier badge ("Researcher · Tier 2"), and a Logout button — and **no** DRF token is present anywhere in the React tree or client bundle.
5. **Given** a verification link older than 48 hours, **when** the user clicks it, **then** `/verify` displays a generic "link invalid or expired" message with a path to request a new one (re-register flow acceptable for MVP).

**Open questions for PM**

- Is "request a new verification link" a Gate 11 surface, or does an expired-link user re-register? Architecture spec doesn't specify; recommend MVP punts to re-register.
- Does `/signup` accept an `institution_id` field at registration? `RegisterSerializer` supports it; the form may or may not expose it.

---

## Story 2 — Coordinator logs in and gets exact map coordinates

**As a** Conservation Coordinator (Tier 3) who was promoted by an operator **I want** to log into the platform with my existing credentials **So that** I see exact (non-generalized) species locations on the map and can use `/dashboard/coordinator` without anyone mailing me a service token

**Acceptance criteria**

1. **Given** a Tier 3 user logs in via Credentials, **when** the JWT callback runs, **then** `tier=3` and `drfToken` are persisted on the JWT and `tierFetchedAt` is set to now.
2. **Given** a logged-in Tier 3 user opens the species map page (SSR), **when** the server component calls `apiFetch` for the `SpeciesLocality` detail endpoint, **then** the request carries `Authorization: Token <drfToken>` and the response contains exact coordinates (not the 0.1-degree generalized form).
3. **Given** a logged-in Tier 3 user navigates to `/dashboard/coordinator`, **when** the page renders, **then** SSR uses `session.drfToken` as the default Authorization source — `COORDINATOR_API_TOKEN` is only consulted if `session.drfToken` is absent or rejected (emergency fallback).
4. **Given** a Tier 3 user was deactivated server-side ( `is_active=False` ), **when** the JWT's 5-minute `/me/` refresh fires on the next request, **then** the 401 clears the session and the user is redirected to `/login` .
5. **Given** a Tier 2 user visits the same map page, **when** the SSR forwards their token, **then** they receive only generalized coordinates — the tier gate is the API's, not the frontend's.

**Open questions for PM**

- The architecture spec proposes the dashboard SSR continues to use `COORDINATOR_API_TOKEN` and flips later; the user's gate scope inverts that. Recommend the PM lock "session-first, service-token-fallback" in the gate spec and document the fallback trigger condition.

---

## Story 3 — Anonymous visitor sees the same public site

**As an** anonymous visitor **I want** the public read surface to behave identically to today **So that** the auth rollout doesn't regress public discovery, SEO, or the ABQ demo path

**Acceptance criteria**

1. **Given** an anonymous visitor lands on `/` , **when** the page renders, **then** no Authorization header is sent on any SSR fetch and all Tier 1 surfaces (species directory, profiles, public dashboard, field programs) render without redirect.
2. **Given** an anonymous visitor opens the map, **when** SSR fetches localities, **then** the request is unauthenticated and only generalized coordinates are returned — identical to current behavior.
3. **Given** the `NEXT_PUBLIC_FEATURE_AUTH` flag is **off**, **when** the visitor browses the site, **then** Login and Sign Up nav links are not rendered, but `/login` and `/signup` direct URLs still resolve for UAT.
4. **Given** an anonymous visitor tries to load `/account` or `/dashboard/coordinator` , **when** middleware evaluates the request, **then** they are redirected to `/login?callbackUrl=…` with the original path preserved on the allow-list.

---

## Story 4 — Operator promotes a Tier 2 to Tier 3

**As an** administrator (Tier 5) **I want** to promote a verified Researcher to Conservation Coordinator from Django admin **So that** institutional partners get coordinator access without the frontend shipping a self-promotion path

**Acceptance criteria**

1. **Given** an operator with Django admin access opens a `User` detail page, **when** they change `access_tier` from 2 to 3 and save, **then** the change is recorded in the audit log with actor, timestamp, and prior value.
2. **Given** the promoted user has an active session, **when** their next request triggers the JWT's 5-minute `/me/` refresh, **then** `token.tier` updates to 3 and tier-gated UI surfaces (coordinator dashboard, exact-coords on map) become available without requiring re-login.
3. **Given** an operator demotes a coordinator from 3 to 2, **when** the next `/me/` refresh fires, **then** the user's tier drops to 2 within five minutes and any in-flight tier-3 SSR fetch returns generalized data on the next request.
4. **Given** an operator deactivates a user ( `is_active=False` ), **when** the next `/me/` refresh fires, **then** `/me/` returns 401, NextAuth clears the JWT, and the user is forced back to `/login` .

**Open questions for PM**

- Should tier changes trigger an email to the affected user? Architecture spec is silent; recommend deferring to a future ops polish ticket.

---

## Story 5 — Existing service-token caller keeps working

**As an** SSR pathway or operations script that authenticates with `COORDINATOR_API_TOKEN` **I want** my requests to keep working unchanged **So that** the auth rollout has a safety valve if NextAuth misbehaves the week before ABQ

**Acceptance criteria**

1. **Given** an SSR call presents `Authorization: Bearer ${COORDINATOR_API_TOKEN}` (no user session), **when** it hits a `TierOrServiceTokenPermission` endpoint, **then** the existing service-token branch authenticates and returns the same data shape as before this gate.
2. **Given** the feature flag `NEXT_PUBLIC_FEATURE_AUTH` is off, **when** any SSR coordinator-scoped fetch runs, **then** the service token is the sole auth source and no NextAuth session lookup is performed (fail-safe for emergency rollback).
3. **Given** both a session DRF token and the service token are available server-side, **when** SSR composes the Authorization header, **then** the user session token wins (default) and the service token is used only when session retrieval fails or returns no `drfToken` .
4. **Given** an existing test or smoke suite that asserts service-token success, **when** the suite runs after this gate ships, **then** assertions still pass without modification.

---

## Cross-cutting acceptance criteria

These apply across every story above. They exist already on the Django side; this gate's job is to verify the frontend doesn't undermine them.

- **Rate limiting.** `/api/v1/auth/login/` enforces 5 failed attempts per 15-minute window per IP via `_check_and_record_rate_limit` . NextAuth's `authorize()` returns `null` on both 401 and 429, surfacing identical generic UI copy ("Invalid email or password"). PM should add a test that documents the 429 response is not leaked through any frontend path. Register and verify endpoints should reuse the same rate-limit primitive keyed differently — flag for PM whether that backend tightening lives in this gate or the next.
- **Account enumeration resistance.** Register, login, and verify all return identical generic responses for missing-vs-wrong-password-vs-unverified. The frontend never branches UI on the specific

failure reason. The signup interstitial does not reveal whether the email was already registered.

- **Cookie security.** In prod the session cookie is `__Secure-next-auth.session-token` with `HttpOnly; Secure; SameSite=Lax; Path=/; Domain=.malagasyfishes.org`. In dev the NextAuth defaults are used and not overridden. `SameSite=Lax` (not Strict) is required so the api subdomain SSR can read the cookie.
- **Session timeout.** JWT session lifetime is 30 days rolling — every request that triggers the `/me/` refresh path also extends the session expiry. Idle sessions older than 30 days re-prompt for credentials.
- **Logout.** `POST /api/v1/auth/logout/` deletes the user's DRF Token (already wired in `views.logout`). NextAuth's `signOut()` clears the session cookie. Both must fire on the Logout action — a logout that clears only the cookie but leaves the DRF Token live is a bug.
- **DRF token never reaches the browser.** The token lives on the JWT (server-only) and is read only inside server components or NextAuth callbacks. No client component, hook, or serialized prop may carry it. This is a security review gate, not a unit-test concern.
- **Open-redirect safety.** `callbackUrl` on `/login` and any post-verify redirect target validate against a `[FRONTEND_BASE_URL, "/"]`-prefix allow-list before redirect.

## Open questions deferred to PM gate breakdown

1. Coordinator dashboard auth source: session-first-with-service-token-fallback (user's gate scope) vs. service-token-still-default (architecture spec §5). Recommend the user's reading.
2. Expired-verification-link recovery: re-register vs. dedicated "resend verification" endpoint. Recommend re-register for MVP.
3. Whether `/signup` exposes the optional `institution_id` field this gate.
4. Email deliverability vendor decision (Mailgun/Resend/SendGrid) — Appendix A of architecture spec flags this as blocking sign-off. Not a story, but a gate-level dependency the PM must call out.
5. Tier-change notification emails — defer.